

Semantic Understanding and Command Following

Semantic Understanding and Command Following

- 1. Course Content
- 2. Preparation
 - 2.1 Content Description
 - 2.2 Starting the Agent
- 3. Running Examples
 - 3.1 Starting the Program
 - 3.2 Test Cases
 - 3.2.1 Case 1
 - 3.2.2 Case 2

1. Course Content

Run the example program and interact with the robot only through text in the terminal, without voice input or voice response.

2. Preparation

2.1 Content Description

This section of the course uses the Jetson Orin NX as an example. For Raspberry Pi and Jetson Nano boards, you need to open a terminal on the host machine and then enter the command to enter the Docker container. After entering the Docker container, enter the commands mentioned in this section of the course in the terminal. For instructions on entering the Docker container from the host machine, please refer to the "Entering the Robot's Docker (For Jetson Nano and Raspberry Pi 5 Users)" section in the product tutorial [0. Instructions and Installation Steps]. For Orin and NX boards, simply open the terminal and enter the commands mentioned in this section of the course.

2.2 Starting the Agent

Note: If the agent is already running, you do not need to start it again.

Enter the following command in the vehicle's terminal:

```
sh start_agent.sh
```

The terminal will print the following information, indicating a successful connection:

```
Jetson@yahboom: ~  
jetson@yahboom: ~ - 177x26  
jetson@yahboom:~$ ./open_agent.sh  
[1749538760.063253] Info | TermiosAgentLinux.cpp | init | running... | fd: 3  
[1749538760.064012] Info | Root.cpp | set_verbose_level | logger setup | verbose_level: 4  
[1749538760.941420] Info | Root.cpp | create_client | create | client_key: 0x0DA64EFC, session_id: 0x81  
[1749538760.941537] Info | SessionManager.hpp | establish_session | session established | client_key: 0x0DA64EFC, address: 0  
[1749538760.979971] Info | ProxyClient.cpp | create_participant | participant created | client_key: 0x0DA64EFC, participant_id: 0x000(1)  
[1749538760.983835] Info | ProxyClient.cpp | create_topic | topic created | client_key: 0x0DA64EFC, topic_id: 0x000(2), participant_id: 0x000(1)  
[1749538760.988244] Info | ProxyClient.cpp | create_publisher | publisher created | client_key: 0x0DA64EFC, publisher_id: 0x000(3), participant_id: 0x000(1)  
[1749538760.993117] Info | ProxyClient.cpp | create_datawriter | datawriter created | client_key: 0x0DA64EFC, datawriter_id: 0x000(5), publisher_id: 0x000(3)  
[1749538760.997675] Info | ProxyClient.cpp | create_topic | topic created | client_key: 0x0DA64EFC, topic_id: 0x001(2), participant_id: 0x000(1)  
[1749538761.001121] Info | ProxyClient.cpp | create_publisher | publisher created | client_key: 0x0DA64EFC, publisher_id: 0x001(3), participant_id: 0x000(1)  
[1749538761.007277] Info | ProxyClient.cpp | create_datawriter | datawriter created | client_key: 0x0DA64EFC, datawriter_id: 0x001(5), publisher_id: 0x001(3)  
[1749538761.010634] Info | ProxyClient.cpp | create_topic | topic created | client_key: 0x0DA64EFC, topic_id: 0x002(2), participant_id: 0x000(1)  
[1749538761.014296] Info | ProxyClient.cpp | create_publisher | publisher created | client_key: 0x0DA64EFC, publisher_id: 0x002(3), participant_id: 0x000(1)  
[1749538761.020171] Info | ProxyClient.cpp | create_datawriter | datawriter created | client_key: 0x0DA64EFC, datawriter_id: 0x002(5), publisher_id: 0x002(3)  
[1749538761.023939] Info | ProxyClient.cpp | create_topic | topic created | client_key: 0x0DA64EFC, topic_id: 0x003(2), participant_id: 0x000(1)  
[1749538761.029173] Info | ProxyClient.cpp | create_publisher | publisher created | client_key: 0x0DA64EFC, publisher_id: 0x003(3), participant_id: 0x000(1)  
[1749538761.034377] Info | ProxyClient.cpp | create_datawriter | datawriter created | client_key: 0x0DA64EFC, datawriter_id: 0x003(5), publisher_id: 0x003(3)  
[1749538761.038946] Info | ProxyClient.cpp | create_topic | topic created | client_key: 0x0DA64EFC, topic_id: 0x004(2), participant_id: 0x000(1)  
[1749538761.042215] Info | ProxyClient.cpp | create_publisher | publisher created | client_key: 0x0DA64EFC, publisher_id: 0x004(3), participant_id: 0x000(1)  
[1749538761.048422] Info | ProxyClient.cpp | create_datawriter | datawriter created | client_key: 0x0DA64EFC, datawriter_id: 0x004(5), publisher_id: 0x004(3)  
[1749538761.051660] Info | ProxyClient.cpp | create_topic | topic created | client_key: 0x0DA64EFC, topic_id: 0x005(2), participant_id: 0x000(1)  
[1749538761.057494] Info | ProxyClient.cpp | create_publisher | publisher created | client_key: 0x0DA64EFC, publisher_id: 0x005(3), participant_id: 0x000(1)  
[1749538761.062183] Info | ProxyClient.cpp | create_datawriter | datawriter created | client_key: 0x0DA64EFC, datawriter_id: 0x005(5), publisher_id: 0x005(3)  
[1749538761.066842] Info | ProxyClient.cpp | create_topic | topic created | client_key: 0x0DA64EFC, topic_id: 0x006(2), participant_id: 0x000(1)  
[1749538761.071217] Info | ProxyClient.cpp | create_subscriber | subscriber created | client_key: 0x0DA64EFC, subscriber_id: 0x000(4), participant_id: 0x000(1)
```

3. Running Examples

3.1 Starting the Program

- Open a terminal on the vehicle's system and enter the following command:

```
ros2 launch multi_brains llm_agent_control.launch.py text_chat_mode:=True
```

- Start the text interaction terminal. This can be started on either the vehicle's system or the virtual machine; **choose only one** method, do not start it on both the virtual machine and the vehicle's system:

```
ros2 run text_chat text_chat
```

3.2 Test Cases

Here are some example test cases. Users can create their own dialogue commands.

- Please move forward 1 meter quickly, then slowly move backward 0.5 meters like a turtle, then turn left 30 degrees, turn right 90 degrees, then translate left 0.5 meters, and then translate right 10 centimeters.
- Please perform a dance, and then tell me a joke about cats and dogs.

3.2.1 Case 1

Open a terminal in the virtual machine, enter the test case in the terminal, and after the model thinks, the AI agent will reply to the user and perform the actions according to the user's instructions.

```
kaiser@kaiser-Redmi-Book: ~/work/yahboom_m3-release1.0
kaiser@kaiser-Redmi-Book: ~/work/yahboom_m3-release1.0 108x19
kaiser@kaiser-Redmi-Book:~/work/yahboom_m3-release1.0$ ros2 run text_chat text_chat
user input: 请你先快速前进1米，然后像乌龟一样慢慢的后退0.5米，然后左转30度，右转90度，然后向左平移0.5米，再向右平移10厘米
okay😊, let me think for a moment... \[INFO] [1749267254.062050173] [text_chat_node]: robot response: "action": ['set_cmdvel(0.5, 0, 0, 2)', 'set_cmdvel(-0.1, 0, 0, 5)', 'move_left(30, 1.5)', 'move_right(90, 1.5)', 'set_cmdvel(0, 0.5, 0, 1)', 'set_cmdvel(0, -0.1, 0, 1)'], "response": 好的，我这就按照你的指令来操作，先快速前进再慢慢后退，然后左转右转，最后左右平移，就像在跳一支复杂的舞蹈一样！
user input: [INFO] [1749267271.304050998] [text_chat_node]: robot response: "action": ['finishtask()'], "response": 我已经完成了所有的动作，感觉就像完成了一次精彩的舞蹈表演！如果你还有其他任务需要我帮忙，尽管告诉我哦！
user input: 结束当前任务
okay😊, let me think for a moment... -[INFO] [1749267424.767042291] [text_chat_node]: robot response: "action": ['finish_dialogue()'], "response": 好的，任务已经结束了。有需要再叫我吧！
user input: 
```

3.2.2 Case 2

Using the same method as Case 1, enter Case 2 in the terminal. The model will reply and perform the actions according to the instructions.

```
kaiser@kaiser-Redmi-Book: ~/work/yahboom_m3-release1.0
kaiser@kaiser-Redmi-Book: ~/work/yahboom_m3-release1.0 108x19
kaiser@kaiser-Redmi-Book:~/work/yahboom_m3-release1.0$ ros2 run text_chat text_chat
user input: 请你先表演个舞蹈，然后再给我讲个关于小猫和小狗的笑话
okay😊, let me think for a moment... -[INFO] [1749267641.452646004] [text_chat_node]: robot response: "action": ['dance()'], "response": 好的，我先来一段舞蹈给你看，准备好欣赏我的舞姿了吗？
user input: [INFO] [1749267656.690005945] [text_chat_node]: robot response: "action": [], "response": 舞蹈结束啦，接下来给你讲个关于小猫和小狗的笑话。为什么小猫和小狗总是吵架？因为它们总是在讨论谁是更好的‘鼠’标！（哈哈，这个笑话有点冷哦~）
[INFO] [1749267660.185565847] [text_chat_node]: robot response: "action": ['finishtask()'], "response": 我已经完成全部任务了，如果你还有其他需要帮忙的，请随时告诉我哦！
结束当前任务
okay😊, let me think for a moment... /[INFO] [1749267682.278947449] [text_chat_node]: robot response: "action": ['finish_dialogue()'], "response": 好的，任务已经结束了，有需要再叫我吧！
user input: 
```